```
import pandas as pd


def value_iteration():
    states = ["S0", "S1"]
    actions = ["Up", "Down", "Left", "Right"]
    gamma = 0.9
    theta = 1e-6

    transitions = {
        "S0": {"Up": "S0", "Down": "S1", "Left": "S0", "Right": "S0"},
        "S1": {"Up": "S0", "Down": "S1", "Left": "S1", "Right": "G"},
    }

    def get_reward(next_state):
        return 100.0 if next_state == "G" else -1.0

    V = {"S0": 0.0, "S1": 0.0, "G": 0.0}

    iteration_history = []
    iteration = 0

    print("================================================")
    print("      MDP DEFINITION & INITIAL SETUP             ")
    print("================================================")
    print(f"States: {states} + ['G' (Terminal)]")
    print(f"Actions: {actions}")
    print(f"Discount Factor (gamma): {gamma}")
    print(f"Initial Values: {V}\n")

    while True:
        iteration += 1
        delta = 0.0
        new_V = V.copy()

        for s in states:
            q_values = {}
            for a in actions:
                next_state = transitions[s][a]
                reward = get_reward(next_state)
                q_val = reward + gamma * V[next_state]
                q_values[a] = round(q_val, 4)

            best_val = max(q_values.values())
            delta = max(delta, abs(best_val - V[s]))
            new_V[s] = best_val

        iteration_history.append(
            {
```

```
                "Iteration": iteration,
                "V(S0)": round(new_V["S0"], 4),
                "V(S1)": round(new_V["S1"], 4),
                "Delta": round(delta, 6),
            }
        )

        V = new_V

        if delta < theta:
            break

    df_iterations = pd.DataFrame(iteration_history)
    print("================================================")
    print("      ITERATION-WISE STATE-VALUE TABLE          ")
    print("================================================")
    print(df_iterations.to_string(index=False))
    print()

    q_table = []
    optimal_policy = {}

    for s in states:
        best_a = None
        best_q = float("-inf")
        for a in actions:
            next_state = transitions[s][a]
            reward = get_reward(next_state)
            q_val = reward + gamma * V[next_state]

            q_table.append(
                {
                    "State": s,
                    "Action": a,
                    "Next State": next_state,
                    "Reward": reward,
                    "Q*(s, a)": round(q_val, 4),
                }
            )

            if q_val > best_q:
                best_q = q_val
                best_a = a

        optimal_policy[s] = best_a

    df_q = pd.DataFrame(q_table)

    print("================================================")
    print("         FINAL OPTIMAL Q-VALUE TABLE            ")
    print("================================================")
    print(df_q.to_string(index=False))
```

```
    print()

    print("=================================================")
    print("            FINAL OPTIMAL RESULTS                ")
    print("=================================================")
    print(f"Optimal State Values V*(s):")
    print(f"  V*(S0) = {V['S0']}")
    print(f"  V*(S1) = {V['S1']}")
    print(f"  V*(G)  = {V['G']} (Terminal)")
    print("\nOptimal Policy pi*(s):")
    for s, a in optimal_policy.items():
        print(f"  pi*({s}) = {a}")

    print(
        f"\nOptimal Path: S0 -> {optimal_policy['S0']} -> S1 -> {optimal_policy['S1']} -> G"
    )


if __name__ == "__main__":
    value_iteration()
```

```
================================================
      MDP DEFINITION & INITIAL SETUP
================================================
States: ['S0', 'S1'] + ['G' (Terminal)]
Actions: ['Up', 'Down', 'Left', 'Right']
Discount Factor (gamma): 0.9
Initial Values: {'S0': 0.0, 'S1': 0.0, 'G': 0.0}

================================================
      ITERATION-WISE STATE-VALUE TABLE
================================================
 Iteration  V(S0)  V(S1)  Delta
         1   -1.0  100.0  100.0
         2   89.0  100.0   90.0
         3   89.0  100.0    0.0

================================================
         FINAL OPTIMAL Q-VALUE TABLE
================================================
State Action Next State  Reward  Q*(s, a)
   S0     Up         S0    -1.0      79.1
   S0   Down         S1    -1.0      89.0
   S0   Left         S0    -1.0      79.1
   S0  Right         S0    -1.0      79.1
   S1     Up         S0    -1.0      79.1
   S1   Down         S1    -1.0      89.0
   S1   Left         S1    -1.0      89.0
   S1  Right          G   100.0     100.0

================================================
            FINAL OPTIMAL RESULTS
================================================
Optimal State Values V*(s):
  V*(S0) = 89.0
  V*(S1) = 100.0
  V*(G)  = 0.0 (Terminal)
```

```
Optimal Policy pi*(s):
  pi*(S0) = Down
  pi*(S1) = Right

Optimal Path: S0 -> Down -> S1 -> Right -> G
```